

U M B R A

PROJECT UPDATE

Design Research and Application Foundation

A summary of the research, technology selection, design system, and
initial application build completed to date.

PREPARED FOR

Brad

DATE

20 July 2026

STATUS

In Development

What This Update Covers

This document summarizes the work completed on the Umbra application since project start. It covers the research behind the product decisions, the reasoning behind the technology stack, the visual design system, and the screens built and verified so far on a live iOS simulator. It closes with the plan for the next phase of work.

§	SECTION
---	---------

01	Research and Requirements Analysis
----	------------------------------------

02	Technology Selection, Why Flutter
----	-----------------------------------

03	Technology Stack and Application Architecture
----	---

04	Visual Design System
----	----------------------

05	Screens Delivered, Simulator Build
----	------------------------------------

06	Next Phase, Roadmap
----	---------------------

Executive Summary

The first phase of Umbra focused on turning the product brief into a validated set of requirements, selecting a technology stack suited to the product's goals, and building a working design system rather than a set of disconnected mockups. That foundation is now expressed in a functioning application, running today on an iOS simulator, covering the first entry points a user experiences: brand introduction, onboarding, and the home and history screens. Every screen shown in this document is live, interactive Flutter code, not a static mockup.

1 Research and Requirements Analysis

Before any screen was designed, the product brief and the hardware feasibility documentation were read together and reconciled into a single working reference, so that design and engineering decisions trace back to a validated source rather than assumption. Where the two source documents disagreed, the most recent client direction was treated as authoritative and the earlier draft superseded.

Product Concept

Umbra pairs a wellness wristband with a companion app built around one simple loop: the band buzzes, the user drinks water. The experience is intentionally passive. There is no manual logging and no setup required for the band's core reminder function to work, the app enhances the experience with live data, history, and adjustable intervals, but never gates it.

Two Reminder Modes

MODE	DEFAULT INTERVAL	ADJUSTABLE OPTIONS	USE CASE
Night Mode	35 minutes	20 / 30 / 45 / 60 min	Active while out for the evening
Daily Mode	90 minutes	20 / 30 / 45 / 60 min	Passive, all day hydration

Confirmed against the client's most recent direction. An earlier draft specified a 40 minute Night default with a different option set, superseded by later correspondence.

Sensor Scope for the First Release

Heart rate is the essential signal, with heart rate variability, respiration rate, skin temperature, SpO2, and accelerometer data all captured to support the experience. Step and sleep data are logged passively and intentionally kept out of the primary interface for this release, in keeping with the product's "ruthlessly simple" direction.

Visual Direction

"Dark background, premium feel, clean and minimal. Think Oura app meets nightlife. Not medical looking."

Client direction, used as the starting brief for the visual design system in Section 04.

2 Technology Selection: Why Flutter

Flutter was selected as the application framework for Umbra. The reasoning is summarized below for transparency, since it is a foundational decision that shapes everything built on top of it.

- **One codebase, both platforms.** Flutter compiles the same source into native iOS and Android applications. If the Dubai launch expands beyond iOS, no second build is required from scratch.
- **Genuinely native performance.** Flutter compiles ahead of time to native ARM machine code rather than running through a bridge. For an interaction heavy, animation rich product like Umbra, this matters directly.
- **Right to left support built into the framework.** Umbra's Dubai market requires a bilingual Arabic and English experience. Flutter's layout system supports right to left direction as a first class citizen, which materially reduces the cost of the localization work planned for the next phase.
- **A mature Bluetooth Low Energy ecosystem.** The wristband communicates over BLE. Flutter's community packages for BLE are production proven and offer the same practical capability as writing directly against each platform's native Bluetooth stack, without maintaining two separate implementations.
- **Faster, safer design iteration.** Flutter's hot reload lets every spacing, color, and motion decision be checked live against the design system while it is being built. For a product where restraint and visual consistency are the entire value proposition, that tight feedback loop is what keeps the app from drifting off brand screen by screen.
- **Lower long term cost of ownership.** A single team and a single codebase is materially cheaper to maintain and extend than two parallel native codebases, without giving up a native feeling result.

This decision was made deliberately rather than by default, and is documented here so it is a visible, considered choice rather than an assumption.

3 Technology Stack and Architecture

The application is built on a small, deliberately curated set of dependencies. Each one was chosen for a specific, current need rather than added speculatively.

```
pubspec.yaml

dependencies:
  flutter:
    sdk: flutter      # core framework

  google_fonts:    ^8.2.0    # Fraunces / Inter typefaces
  provider:        ^6.1.5    # app state management
  cupertino_icons: ^1.0.8    # iOS style icon set

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints:   ^6.0.0    # enforced code style rules
```

Current, active dependencies. Bluetooth connectivity and hardware integration packages are scoped for the next phase, see Section 06.

Design Tokens in Practice

Every color, spacing value, and radius used anywhere in the app is a named constant, never a value typed directly into a screen. This is what keeps the interface visually consistent as more screens are added.

```
lib/core/constants/app_colors.dart

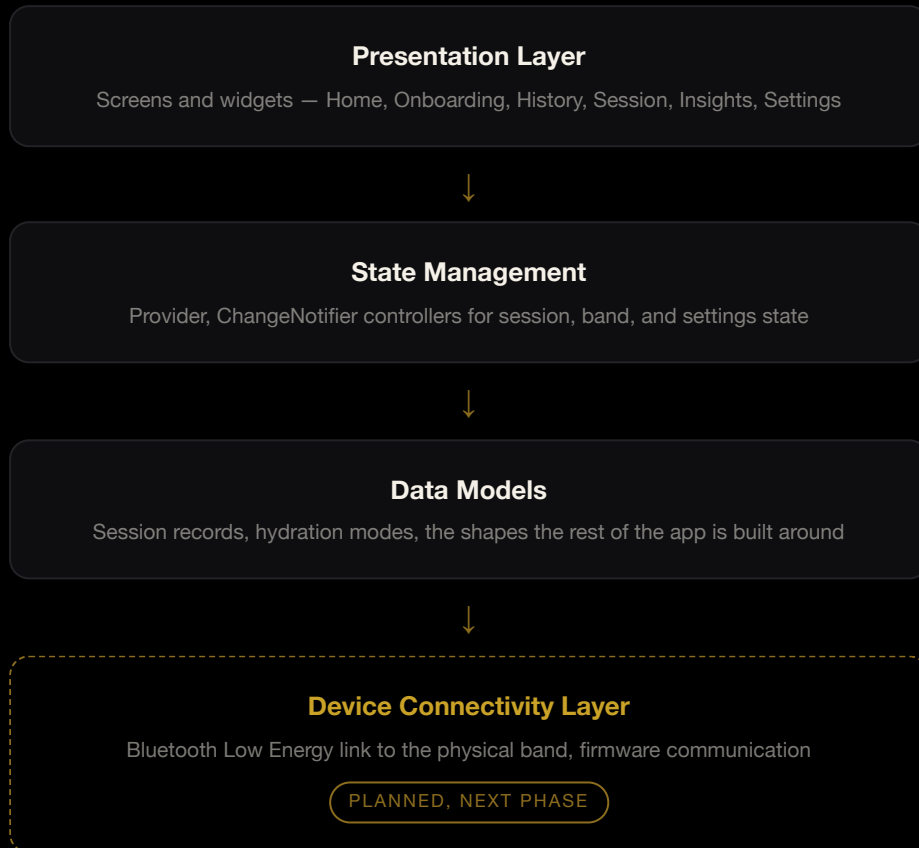
class AppColors {
  // Background
  static const Color bgVoid      = Color(0xFF000000);
  static const Color bgSurface = Color(0xFF0E0E10);

  // Gold accent, spent deliberately
  static const Color gold        = Color(0xFFC9A227);
  static const Color champagne = Color(0xFFE8D9A0);
}
```

Excerpt. No raw color value appears anywhere else in the codebase, every screen reads from this single source.

Application Architecture

The application is layered so that the interface, the app's state, and the eventual hardware connection are cleanly separated. Each layer only knows about the one directly beneath it.

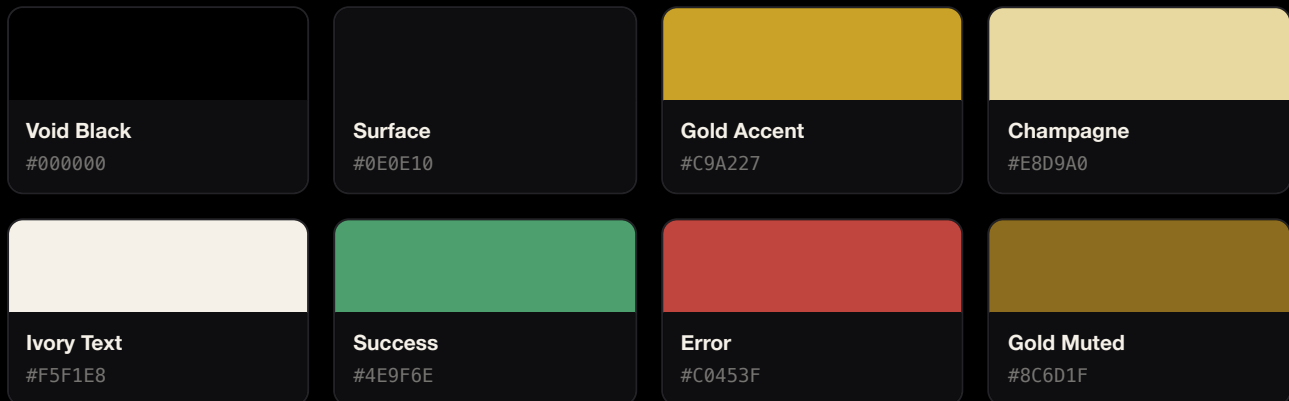


This separation means the connectivity layer can be built and tested independently, then connected in, without reworking the screens or the state layer already in place.

Visual Design System

Rather than styling each screen on its own, every visual property in the app draws from one shared system. This is what makes the product feel considered rather than assembled, and it is what makes the client's own brief, "dark background, premium feel, clean and minimal", enforceable across every screen rather than a one time styling pass.

Color Palette



A single accent color, used deliberately. Gold marks the one thing on a screen meant to draw the eye, not every icon and label.

Typography

ROLE	TYPEFACE	USED FOR
Display	Fraunces	Brand moments, greetings, hero numerals only
Body / UI	Inter	Every other line of text in the app

Arabic counterparts (Amiri for display, IBM Plex Sans Arabic for body) are specified for the bilingual localization pass in Section 06, matched in weight and character to their Latin pairing.

Motion and Feedback

Every tappable element responds to touch. Motion is restrained by rule, four durations only, nothing bounces or spins without a state change to justify it. This keeps the app feeling confident rather than eager.

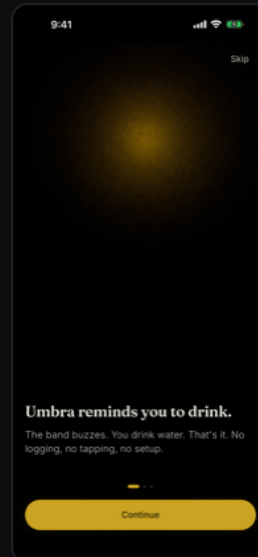
5 Screens Delivered

The screens below are running Flutter code, captured directly from an iOS simulator, not static design mockups. They represent the brand entrance and the first two tabs of the application.



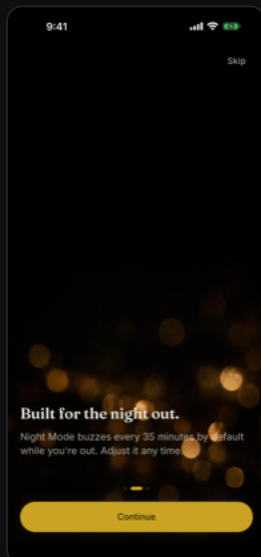
Brand Entrance

Splash screen, the app's one deliberate brand moment



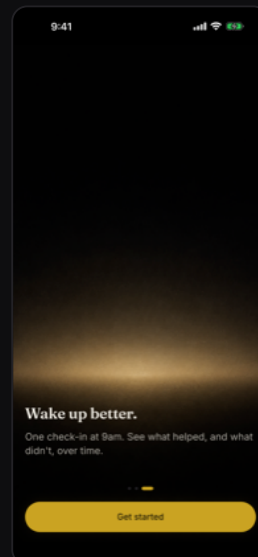
Onboarding, 1 of 3

Introduces the core loop: the band buzzes, you drink



Onboarding, 2 of 3

Introduces Night Mode and the default reminder interval

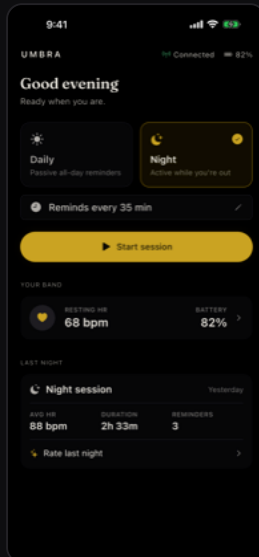


Onboarding, 3 of 3

Introduces the single morning check in

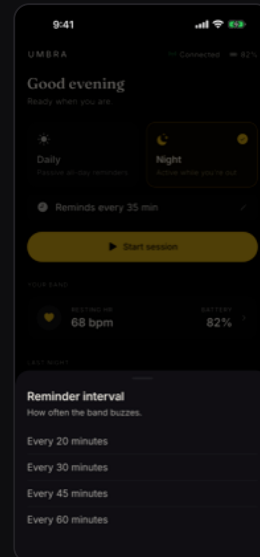
Home and History

Home is the app's center of gravity: mode selection, the reminder interval, band status, and the most recent session at a glance. History lists past sessions with their biometric averages.



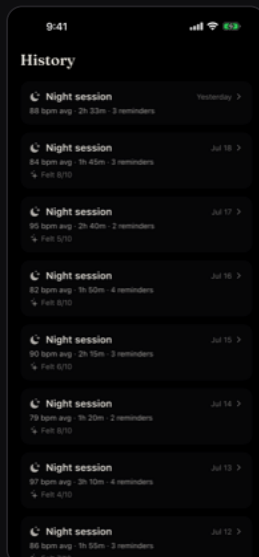
Home

Mode selection, reminder interval, band status, last session



Home, Interval Selector

Tapping the reminder row opens the interval picker



History

Past sessions, each with duration, average heart rate, and reminders sent

Band status and session data shown above use representative values while the physical prototype is in build. The interface and data flow are real, the same screens will read live values once the hardware connection lands, see Section 06.

6 Next Phase

With the research, technology foundation, design system, and first screens in place, the next phase of work is scoped as follows.

ITEM	DESCRIPTION
1 Insights Screen	An analytical view surfacing the patterns behind the best and worst mornings, in the same restrained visual language as the rest of the app.
2 Settings Screen	Account, band, and preference controls.
3 Firmware Connectivity	The Bluetooth Low Energy layer connecting the app to the physical band, built against the connectivity contract shown in Section 03.
4 Hardware Integration Packages	The Bluetooth and permission handling packages required once the connectivity layer work begins.
5 Live Hardware Testing	End to end testing against the physical band prototype once it arrives, validating the reminder loop and live biometric data on real hardware.
6 Bilingual Localization	Arabic and English throughout, including full right to left layout support, ahead of the Dubai launch.
7 Companion Wearable App	A watch companion mirroring the active session and firing reminders on the wrist.

Item sequencing may adjust based on hardware prototype availability. Firmware and live hardware testing are dependent on the physical band arriving from the hardware side of the project.

U M B R A

Thank you

This document reflects the state of the project as of 20 July 2026.
Questions on any section are welcome at any time.

Confidential, prepared for internal project review.